
WEB TECHNOLOGY

(24BECSE304)

Module 4

Topics: Dynamic Documents with JavaScript & Welcome to React

PREPARED BY:
Prof. Shruthi V Kulkarni
Assistant Professor
Dept. of CSE, SNPSU

Module 4

Chapter 1

Dynamic Documents with JavaScript

Introduction

DHTML is not a new markup language. It is a collection of technologies that allow dynamic changes to HTML documents. Enables modification of:

- Tag attributes
- Tag contents
- Element style properties

Changes occur **after** the document has been loaded and is still being displayed. Triggered by:

- User interaction (e.g., clicking a button, hovering over text)
- Browser events (e.g., page load, resize)

Implemented using:

- Embedded scripts (JavaScript)
- Accessing HTML elements via the DOM (Document Object Model)

Browser Support for DHTML is not uniform across browsers & this discussion follows W3C-standard approaches (not vendor-specific features).

Event Models in Use are:

- DOM 0 Event Model:
 - Used in most examples.
 - Works on both Internet Explorer 8 (IE8) and Firefox 3 (FX3).
- DOM 2 Event Model:
 - Used in special cases (e.g., example in Section 11).
 - Cannot be implemented in a standard way with DOM 0 in some situations.
 - IE8 and earlier: Does not support DOM 2 Event Model.
 - IE9 and later: Supports DOM 2 Event Model.

Key Points to Remember

- DHTML = HTML + CSS + JavaScript + DOM (together for dynamic interaction).
- Changes happen in real-time without reloading the page.
- Compatibility issues exist between browsers, especially older ones.

Positioning Elements

Early web design offered little control over the placement of HTML elements. Elements were arranged sequentially (like in a word processor): Fill a row → Start a new row → fill it → so on.

HTML tables provided a basic column framework for layout:

- Helped arrange elements but were inflexible.
- Slow to display large or complex layouts.

CSS-P (Cascading Style Sheets – Positioning) was released by W3C in 1997 with a purpose of Precise control over element positioning & has browser Support : IE8, IE9, Firefox 3, Chrome 12 (full support). CSS-P allows positioning of any element anywhere on a web page display. It also supports dynamic repositioning of elements using JavaScript by changing positioning-related style properties.

Key Style Properties:

1. **left** – Specifies the distance from the left reference point to the element's position.
2. **top** – Specifies the distance from the top reference point to the element's position.
3. **position** – Works with left and top to control placement and movement.
 - **Values:**
 - **absolute** – Positions the element exactly relative to its first positioned ancestor.
 - **relative** – Positions the element relative to its normal position.
 - **static** – Default value; elements are positioned according to the normal flow of the page.

Dynamic Movement:

- With JavaScript, you can change **left** and **top** values in real time, enabling animation or interaction.

Absolute Positing

Absolute positioning places an element at a specific location on the page, ignoring the positions of other elements. Coordinates are defined relative to the nearest positioned ancestor (or the document body if none exists).

Syntax:

```
<p style="position: absolute; left: 100px; top: 200px">
    -- text --
</p>
```

Where,

- position: absolute; → enables absolute positioning.
- left: 100px; → places the element 100 pixels from the **left** edge of the containing element/window.
- top: 200px; → places the element 200 pixels from the **top** edge of the containing element/window.

Key Uses:

1. Fixed Placement
 - Used when you want an element in an exact position regardless of surrounding content.
2. Layering Elements
 - Can be used to superimpose text over other elements, such as creating a watermark effect.

Watermark Effect Example:

- Special text (e.g., a subliminal message) can be placed over a normal paragraph:

- Larger italicized font.
- Light-gray color for subtlety.
- Increased letter spacing for readability.
- Both ordinary and special text remain legible.

Font Sizing with em:

- em is a relative unit.
- Size is relative to the parent element's font size.
- Ensures text scales proportionally in nested elements.

For example, a paragraph of normal text that describes apples is displayed. Superimposed on this paragraph is the somewhat subliminal message “APPLES ARE GOOD FOR YOU”. Here is the document: `absPos.html`

```
<!DOCTYPE html>
<!-- absPos.html
Illustrates absolute positioning of elements
-->
<html lang = "en">
<head>
<title> Absolute positioning </title>
<meta charset = "utf-8" />
<style type = "text/css">
/* A style for a paragraph of text */
.regtext {font-family: Times; font-size: 1.2em; width: 500px}
/* A style for the text to be absolutely positioned */
.abstext {position: absolute; top: 25px; left: 25px;
font-family: Times; font-size: 1.9em;
font-style: italic; letter-spacing: 1em;
color: rgb(160,160,160); width: 450px}
</style>
</head>
<body>
<p class = "regtext">
Apple is the common name for any tree of the genus Malus,
of the family Rosaceae. Apple trees grow in any of the
temperate areas of the world. Some apple blossoms are white,
but most have stripes or tints of rose. Some apple blossoms
are bright red. Apples have a firm and fleshy structure that
grows from the blossom. The colors of apples range from
green to very dark red. The wood of apple trees is fine
grained and hard. It is, therefore, good for furniture
construction. Apple trees have been grown for many
centuries. They are propagated by grafting because they
do not reproduce themselves.
<span class = "abstext">
APPLES ARE GOOD FOR YOU
</span>
</p>
</body></html>
```

Display of absPos.html

Apple is the common name for any tree of the genus *Malus*, of the family Rosaceae. Apple trees grow in any of the temperate areas of the world. Some apple blossoms are white, but most have stripes or tints of rose. Some apple blossoms are bright red. Apples have a firm and fleshy structure that grows from the blossom. The colors of apples range from green to very dark red. The wood of apple trees is fine grained and hard. It is, therefore, good for furniture construction. Apple trees have been grown for many centuries. They are propagated by grafting because they do not reproduce themselves.

Width Property Usage:

- Purpose: Ensures that special text is uniformly embedded within regular text.
- Without width:
 - Text would stretch to the right edge of the browser window.
 - The browser window width can vary between clients and can change anytime if the user resizes the window.

Absolute Positioning Inside Another Element:

- If an element is absolutely positioned inside another positioned element (one that has position specified), the top and left values:
 - Are measured from the upper-left corner of the enclosing element.
 - Not from the browser window's upper-left corner.

To illustrate the placement of nested elements, the document absPos.html is modified to place the regular text 100 pixels from the top and 100pixels from the left. The special text is nested inside the regular text by using <div> and tags. The modified document, which is named absPos2.html, is as follows:

```
<!DOCTYPE html>
<!-- absPos2.html
Illustrates nested absolute positioning of elements
-->
<html lang = "en">
<head>
<title> Nested absolute positioning </title>
<meta charset = "utf-8" />
<style type = "text/css">
/* A style for a paragraph of text */
.regtext {font-family: Times; font-size: 1.2em; width: 500px;
position: absolute; top: 100px; left: 100px;}
/* A style for the text to be absolutely positioned */
.abstext {position: absolute; top: 25px; left: 25px;
font-family: Times; font-size: 1.9em;
font-style: italic; letter-spacing: 1em;
color: rgb(160,160,160); width: 450px;}
</style>
</head>
<body>
<p class = "regtext">
Apple is the common name for any tree of the genus Malus,
of the family Rosaceae. Apple trees grow in any of the
temperate areas of the world. Some apple blossoms are white,
but most have stripes or tints of rose. Some apple blossoms
are bright red. Apples have a firm and fleshy structure that
grows from the blossom. The colors of apples range from
green to very dark red. The wood of apple trees is fine
grained and hard. It is, therefore, good for furniture
construction. Apple trees have been grown for many
centuries. They are propagated by grafting because they
do not reproduce themselves.
<span class = "abstext">
APPLES ARE GOOD FOR YOU
</span>
</p>
</body>
</html>
```

Display of absPos2.html

Apple is the common name for any tree of the genus *Malus*, of the family Rosaceae. Apple trees grow in any of the temperate areas of the world. Some apple blossoms are white, but most have stripes or tints of rose. Some apple blossoms are bright red. Apples have a firm and fleshy structure that grows from the blossom. The colors of apples range from green to very dark red. The wood of apple trees is fine grained and hard. It is, therefore, good for furniture construction. Apple trees have been grown for many centuries. They are propagated by grafting because they do not reproduce themselves.

Relative Positioning:

Relative positioning means an element is placed relative to its natural position in the document's flow. When you use relative positioning, an element is positioned relative to where it would regularly be. If the top and left properties are given, then relative positioning displace the element by the specified amount from the natural position. If the top and left properties are not specified then element is positioned as if like the it is statically positioned. However, such an element can be moved later. Relative positioning is used for creating different effects in the document. It can be used to highlight the special words in the text. The following example highlight the word "good" in line of text.

relPos.html

```
<!DOCTYPE html>
<!-- relPos.html
Illustrates relative positioning of elements
-->
<html lang = "en">
<head>
<title> Relative positioning </title>
<meta charset = "utf-8" />
<style type = "text/css">
.regtext {font: 2em Times}
.spectext {font: 2em Times; color: red; position: relative;
top: 15px;}
</style>
</head>
<body>
<p class = "regtext">
Apples are
<span class = "spectext"> GOOD </span> for you.
</p>
</body></html>
```

Display of relPos.html



Relative positioning can be used to create superscripts. For example, the following can be used to place “xyz” 10 pixels above the natural baseline of the text.

```
<p> The superscript in this name <span style="position: relative; top:-3px"> xyz</span> is “xyz”. </p>
```

Static Positioning:

'Static' positioning is identical to normally rendered HTML. These elements cannot be positioned or repositioned, nor do they define a coordinate system for child elements. This is the default value for 'position', except for the <BODY> element, which, while it cannot be positioned, does define a coordinate system for child elements.

Moving Elements:

- Movement Mechanism:
 - Change the `top` or `left` property values.
 - Absolute positioning → Element moves directly to the new `top` and `left` coordinates (relative to its containing block).
 - Relative positioning → Element moves from its original position by the given distances.
- Setup:
 - An image is absolutely positioned on the display.
 - Two text boxes:
 - X coordinate → sets the `left` property.
 - Y coordinate → sets the `top` property.
 - A "Move It" button → triggers the movement.
- Process:
 - User enters new values in the text boxes.
 - On clicking Move It:
 - The image's `top` and `left` values are updated.
 - The element moves to the new position.

JavaScript Function (External File):

- Function changes `top` and `left` property values.
- Parameters Sent:
 1. Element ID → identifies the element to move (demonstrates reusability for multiple elements).
 2. Text box values → passed as DOM addresses with `.value` to retrieve user input.

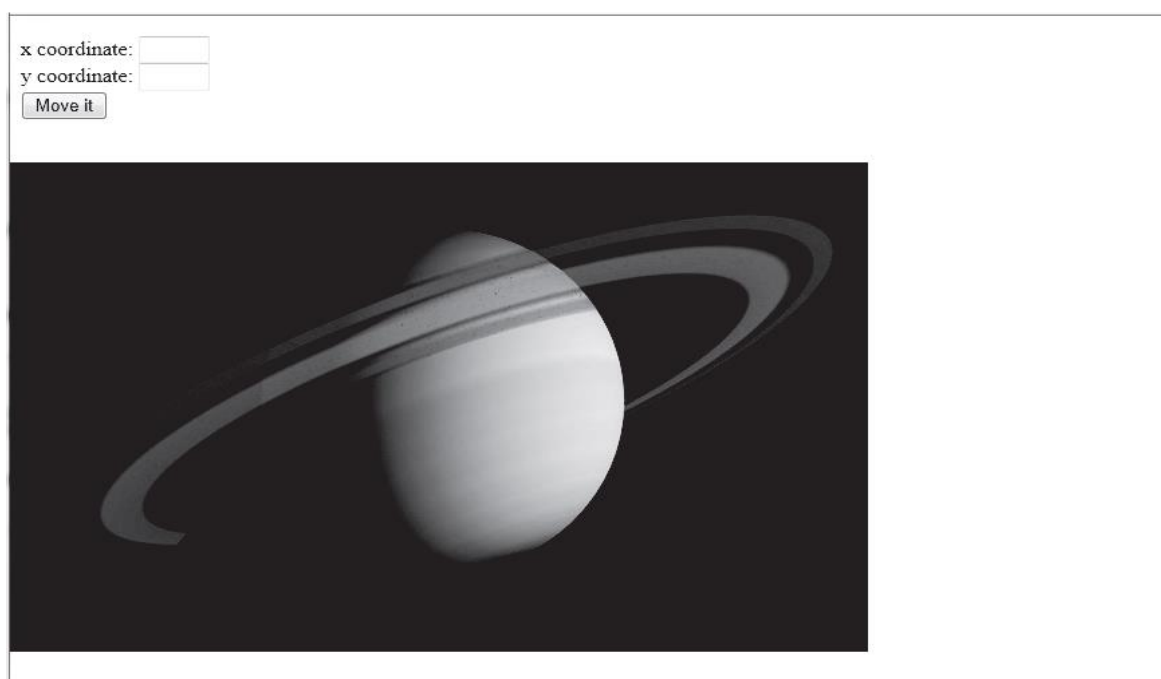
HTML elements with position set to absolute or relative can be moved. Style is used in DOM to change top and left because these are CSS style properties. Input from text boxes → string representation of numbers. top and left properties require unit abbreviations (e.g., "px"). JavaScript concatenates "px" to the numeric input before assignment.

This document, called mover.html, and the associated JavaScript file, mover.js, are as follows:

```
<!DOCTYPE html>
<!-- mover.html
Uses mover.js to move an image within a document
-->
<html lang = "en">
<head>
<title> Moving elements </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "mover.js" >
</script>
</head>
<body>
<form action = "">
<p>
<label>
x coordinate:
<input type = "text" id = "leftCoord" size = "3" />
</label>
<br />
<label>
y coordinate:
<input type = "text" id = "topCoord" size = "3" />
</label>
<br />
<input type = "button" value = "Move it"
onclick =
"moveIt('saturn',
document.getElementById('topCoord').value,
document.getElementById('leftCoord').value)" />
</p>
</form>
<div id = "saturn" style = "position: absolute;
top: 115px; left: 0;">
<img src = "../images/saturn.png"
alt = "(Picture of Saturn)" />
</div>
</body>
</html>
```

```
// mover.js
// Illustrates moving an element within a document
// The event handler function to move an element
function moveIt(movee, newTop, newLeft) {
  dom = document.getElementById(movee).style;
  // Change the top and left properties to perform the move
  // Note the addition of units to the input values
  dom.top = newTop + "px";
  dom.left = newLeft + "px";
}
```

Display of mover.html (before pressing the *Move It* button)



Display of mover.html (after pressing the *Move It* button)



Element Visibility

Document elements can be specified to be visible or hidden with the values of their visibility property. The two possible values for the visibility are – visible and hidden.

showHide.html

```
<!DOCTYPE html>
<!-- showHide.html
Uses showHide.js
Illustrates visibility control of elements
-->

<html lang = "en">
<head>
<title> Visibility control </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "showHide.js" >
</script>
</head>
<body>
<form action = "">
<div id = "saturn" style = "position: relative;
visibility: visible;">
<img src = "../images/saturn.png"
alt = "(Picture of Saturn)" />
</div>
<p>
<br />
<input type = "button" value = "Toggle Saturn"
onclick = "flipImag()" />
</p>
</form>
</body>
</html>
```

```
// showHide.js
// Illustrates visibility control of elements
// The event handler function to toggle the visibility
// of the images of Saturn
function flipImag() {
dom = document.getElementById("saturn").style;
// Flip the visibility adjective to whatever it is not now
if (dom.visibility == "visible")
dom.visibility = "hidden";
else
dom.visibility = "visible";
}
```

Changing Colors and Fonts

The background and foreground colors of the document display and font properties of the text can be changed dynamically

Changing Colors:

Dynamic changes to colors are relatively simple. In the next example, the user is presented with two text boxes into which color specifications can be typed—one for the document background color and one for the foreground color.

The colors can be specified in any of the three ways that color properties can be given in CSS. A JavaScript function that is called whenever one of the text boxes is changed makes the change in the document's appropriate color property: backgroundColor or color.

The first of the two parameters to the function specifies whether the new color is for the background or foreground; the second specifies the new color. The new color is the value property of the text box that was changed by the user.

In this example, the calls to the handler functions are in the HTML text box elements. This approach allows a simple way to reference the element's DOM address.

The JavaScript this variable in this situation is a reference to the object that represents the element in which it is referenced. A reference to such an object is its DOM address.

Therefore, in a text element, the value of this is the DOM address of the text element. So, in the example, this.value is used as an actual parameter to the handler function. Because the call is in an input element, this.value is the DOM address of the value of the input element.

This document, called dynColors.html, and the associated JavaScript file are as follows:

```
<!DOCTYPE html>
<!-- dynColors.html
Uses dynColors.js
Illustrates dynamic foreground and background colors
-->
<html lang = "en">
<head>
<title> Dynamic colors </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "dynColors.js" >
</script>
</head>
<body>
<p style = "font-family: Times; font-style: italic;
font-size: 2em;" >
This small page illustrates dynamic setting of the
foreground and background colors for a document
</p>
<form action = "">
<p>
<label>
Background color:
```

```
<input type = "text" name = "background" size = "10"
onchange = "setColor('background', this.value)" />
</label>
<br />
<label>
Foreground color:
<input type = "text" name = "foreground" size = "10"
onchange = "setColor('foreground', this.value)" />
</label>
<br />
</p>
</form>
</body>
</html>
```

```
// dynColors.js
// Illustrates dynamic foreground and background colors
// The event handler function to dynamically set the
// color of background or foreground
function setColor(where, newColor) {
if (where == "background")
document.body.style.backgroundColor = newColor;
else
document.body.style.color = newColor;
}
```

Changing Fonts:

Web users are accustomed to having links in documents change color when the cursor is placed over them. Any property of a link can be changed by using the mouse event, ‘mouseover’ to trigger JavaScript event handlers. Thus the font style and font size, as well as the color, can be changed when the cursor is placed over a link.

The link can be changed back to its original form when an event handler is triggered with the ‘mouseout’ event. In the following example, the only element is a sentence with an embedded link. The foreground color for the document is the default black. The link is presented in blue. When the mouse cursor is placed over the link, its color changes to red and its font style changes to italic.

dynFont.html

```
<!DOCTYPE html>
<!-- dynFont.html
Illustrates dynamic font styles and colors
-->
<html lang = "en">
<head>
<title> Dynamic fonts </title>
<meta charset = "utf-8" />
<style type = "text/css">
```

```
.regText {font: 1.1em 'Times New Roman';}
.wordText {color: blue;}
</style>
</head>
<body>
<p class = "regText">
The state of
<span class = "wordText";
onmouseover = "this.style.color = 'red';
this.style.fontStyle = 'italic';
this.style.fontSize = '2em';";
onmouseout = "this.style.color = 'blue';
this.style.fontStyle = 'normal';
this.style.fontSize = '1.1em';";>
Washington
</span>
produces many of our nation's apples.
</p>
</body>
</html>
```

Display of dynFont.html with the mouse cursor not over the word



Display of dynFont.html with the mouse cursor over the word



Dynamic Content

- Dynamic changes can be made not only to position, visibility, colors, and styles of elements, but also to the content inside them.
- Content changes are done through the **value** property of the associated JavaScript object.
- Changing content is conceptually similar to changing style properties.
- **Purpose:** Assist users while filling out forms by providing contextual help in a dedicated text area (help box).
- **Behavior:**
 - When the mouse cursor is placed over an input field → help box shows specific guidance for that field.
 - When the cursor moves away from all input fields → help box reverts to a default message (“assistance is available”).
- **JavaScript Array:** Stores all help messages for different fields.
- **Events Used:**
 1. **mouseover** → Calls a handler function to change help box content to the relevant message.

2. **mouseout** → Calls a handler function to revert content to the standard default message.

- **Handler Function:**

- Receives a parameter that specifies which message to display.
- Updates the help box content dynamically.

dynValue.html

```
<!DOCTYPE html>
<!-- dynValue.html
Illustrates dynamic values
-->
<html lang = "en">
<head>
<title> Dynamic values </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "dynValue.js" >
</script>
<style type = "text/css">
textarea {position: absolute; left: 250px; top: 0px;}
span {font-style: italic;}
p {font-weight: bold;}
</style>
</head>
<body>
<form action = "">
<p>
<span>
Customer information
</span>
<br /><br />
<label>
Name:
<input type = "text" onmouseover = "messages(0) "
onmouseout = "messages(4) " />
</label>
<br />
<label>
Email:
<input type = "text" onmouseover = "messages(1) "
onmouseout = "messages(4) " />
</label>
<br /> <br />
<span>
To create an account, provide the following:
</span>
<br /> <br />
<label>
User ID:
<input type = "text" onmouseover = "messages(2) "
onmouseout = "messages(4) " />
</label>
<br />
```

```
<label>
Password:
<input type = "password"
onmouseover = "messages(3)"
onmouseout = "messages(4)" />
</label>
<br />
</p>
<textarea id = "adviceBox" rows = "3" cols = "50">
This box provides advice on filling out the form
on this page. Put the mouse cursor over any input
field to get advice.
</textarea>
<br /><br />
<input type = "submit" value = "Submit" />
<input type = "reset" value = "Reset" />
</form>
</body>
</html>
```

```
// dynValue.js
// Illustrates dynamic values
var helpers = ["Your name must be in the form: \n \
first name, middle initial., last name",
"Your email address must have the form: \
user@domain",
"Your user ID must have at least six characters",
"Your password must have at least six \
characters and it must include one digit",
"This box provides advice on filling out\
the form on this page. Put the mouse cursor over any \
input field to get advice"]
// *****
// The event handler function to change the value of the
// textarea
function messages(adviceNumber) {
document.getElementById("adviceBox").value =
helpers[adviceNumber];
}
```

Display of dynValue.html

Customer information

Name:

Email:

To create an account, provide the following:

User ID:

Password:

This box provides advice on filling out the form on this page. Put the mouse cursor over any input field to get advice

Stacking Elements

The top and left properties used to place an element anywhere within the two-dimensional display of a document. These values define vertical and horizontal positioning. Although the browser display is two-dimensional, a third dimension effect can be created using stacked elements. Similar to stacking windows in a graphical user interface. Multiple elements can occupy the same space. Only one element (the top one) is visible in the overlapping area.

z-index attribute determines which element appears on top in overlapping situations. Higher z-index value → element displayed above elements with lower z-index.

JavaScript equivalent property: `style.zIndex`. Used to bring elements forward or send them back in the stacking order.

stacking.html

```
<!DOCTYPE html>
<!-- stacking.html
Uses stacking.js
Illustrates dynamic stacking of images.
-->
<html lang = "en">
<head>
<title> Dynamic stacking of images </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "stacking.js" >
</script>
<style type = "text/css">
.plane1 {position: absolute; top: 0; left: 0;
z-index: 0;}
.plane2 {position: absolute; top: 50px; left: 110px;
z-index: 0;}
.plane3 {position: absolute; top: 100px; left: 220px;
z-index: 0;}
```

```
</style>
</head>
<body>
<p>
<img class = "plane1" id = "airplane1"
src = "../images/airplane1.png"
alt = "(Picture of an airplane)"
onclick = "toTop('airplane1')" />
<img class = "plane2" id = "airplane2"
src = "../images/airplane2.png"
alt = "(Picture of an airplane)"
onclick = "toTop('airplane2')" />
<img class = "plane3" id = "airplane3"
src = "../images/airplane3.png"
alt = "(Picture of an airplane)"
onclick = "toTop('airplane3')" />
</p>
</body>
</html>
```

```
// stacking.js
// Illustrates dynamic stacking of images
var topp = "airplane1";
// The event handler function to move the given element
// to the top of the display stack
function toTop(newTop) {
// Set the two dom addresses, one for the old top
// element and one for the new top element
domTop = document.getElementById(topp).style;
domNew = document.getElementById(newTop).style;
// Set the zIndex properties of the two elements, and
// reset topp to the new top
domTop.zIndex = "0";
domNew.zIndex = "10";
topp = newTop;
}
```

The initial display of `stacking.html` (photographs courtesy of Cessna Aircraft Company)



The display of `stacking.html` after clicking the second image (photographs courtesy of Cessna Aircraft Company)



The display of `stacking.html` after clicking the bottom image (photographs courtesy of Cessna Aircraft Company)



Locating the Mouse Cursor

When something happens on a web page like you clicking your mouse, moving it around, or pressing a key the browser quietly creates a little “report” about it. This report is called an **event object**, and it contains all the details about what just happened.

If you click somewhere on the page, that click is a special kind of event called a **MouseEvent**. Along with other information, it tells you **where** you clicked in two different ways:

- **clientX** and **clientY**
 - Think of these as the local address inside your browser window.
 - They measure your click's position from the top-left corner of the browser window, in pixels.
 - This is the version you'll use most often.
- **screenX** and **screenY**
 - This is the big-picture address, telling you where you clicked on your actual computer screen.
 - It's less common, but can be handy if you're working with multi-monitor setups or full-screen apps.

The trick is in passing the event object to the JavaScript function that handles the click.

- When you click, the browser automatically creates the event object.
- In Firefox, this object is handed directly to your function if you list it as a parameter.
- In Chrome and Internet Explorer, the object exists as a global variable meaning it's just "there" without you asking for it.

One interesting note about the preceding cursor-finding example is that, with IE and Chrome, the mouse clicks are ignored if the mouse cursor is below the last element on the display. The FX browser always responds the same way, regardless of where the cursor is on the display.

where.html

```
<!DOCTYPE html>
<!-- where.html
Uses where.js
Illustrates x- and y-coordinates of the mouse cursor
-->
<html lang = "en">
<head>
<title> Where is the cursor? </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "where.js" >
</script>
</head>
<body onclick = "findIt(event)">
<form action = "">
<p>
Within the client area: <br />
x:
<input type = "text" id = "xcoor1" size = "4" />
y:
<input type = "text" id = "ycoor1" size = "4" />
<br /><br />
Relative to the origin of the screen coordinate system:
<br />
x:
<input type = "text" id = "xcoor2" size = "4" />
```

```
y:
<input type = "text" id = "ycoor2" size = "4" />
</p>
</form>
<p>
<img src = "../images/airplane1.png" alt = "(Picture of an
airplane)" />
</p>
</body>
</html>
```

```
// where.js
// Show the coordinates of the mouse cursor position
// in an image and anywhere on the screen when the mouse
// is clicked
// The event handler function to get and display the
// coordinates of the cursor, both in an element and
// on the screen
function findIt(evt) {
document.getElementById("xcoor1").value = evt.clientX;
document.getElementById("ycoor1").value = evt.clientY;
document.getElementById("xcoor2").value = evt.screenX;
document.getElementById("ycoor2").value = evt.screenY;
}
```

Display of where.html (the cursor was in the tail section of the plane)



Reacting to a Mouse Click

The **mousedown** event will displays the message *"Please don't click here!"* & the **mouseup** event will hides the message. The effect happens whenever the mouse button is clicked, no matter where the cursor is at the time. The message is shown near the mouse cursor.

- **Offsets applied** to position:
 - **Left offset:** -130 pixels → Moves the message horizontally to center it over the cursor.
 - **Top offset:** -25 pixels → Moves the message vertically to center it.
- **IE & Chrome:**
 - The message only appears if the click happens **within a specific part of the display** defined by
 elements.
- **Firefox:**
 - A click **anywhere on the screen** will trigger the display of the message.

anywhere.html

```
<!DOCTYPE html>
<!-- anywhere.html
Uses anywhere.js
Display a message when the mouse button is pressed,
no matter where it is on the screen
-->
<html lang = "en">
<head>
<title> Sense events anywhere </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "anywhere.js" >
</script>
</head>
<body onmousedown = "displayIt(event);"
onmouseup = "hideIt();">
<p>
<span id= "message"
style = "color: red; visibility: hidden;
position: relative;
font-size: 1.7em; font-style: italic;
font-weight: bold;">
Please don't click here!
</span>
<br /><br /><br /><br /><br /><br /><br /><br />
<br /><br /><br /><br /><br /><br /><br /><br />
</p>
</body>
</html>
```

```
// anywhere.js
// Display a message when the mouse button is pressed,
// no matter where it is on the screen
// The event handler function to display the message
function displayIt(evt) {
var dom = document.getElementById("message");
dom.style.left = (evt.clientX - 130) + "px";
dom.style.top = (evt.clientY - 25) + "px";
dom.style.visibility = "visible";
}

// *****
// The event handler function to hide the message
function hideIt() {
document.getElementById("message").style.visibility =
"hidden";
}
```

Slow Movements of Elements

Instant movement is done by changing `top` and `left` properties directly. Slow movement is achieved by:

- Moving the element in small steps.
- Adding short delays between each step.

a) `setTimeout()`

- Executes code once after a specified delay (in milliseconds).
- Syntax: `setTimeout("mover()", 20);`
→ Calls `mover()` after 20 milliseconds.

b) `setInterval()`

- Executes code repeatedly at a set interval.
- Two forms:
 1. Two parameters:
 - Code (string or function name)
 - Interval (milliseconds)
 2. Multiple parameters:
 - Function name (not as a string)
 - Interval (milliseconds)
 - Additional parameters to pass to the function

In the example, `moveText.html`

- Moves text from (100, 100) to (300, 300).
- Uses `setTimeout()` repeatedly to call a `moveText()` function every millisecond until the destination is reached.

Initial Setup

- Text is placed in a `<span>` element at (100, 100).

- `onload` in `<body>` calls `initText()`:
 - Gets initial `left` and `top` values.
 - Removes `"px"` so values can be used as numbers.
 - Calls `moveText()`.

The `moveText()` Function

- Moves coordinates one pixel at a time toward `(300, 300)`.
- Works in any direction (not just down-right).
- After updating coordinates, calls itself again using `setTimeout()` until the target is reached.
- Before setting the new position, `"px"` is added back to the numeric values.

CSS position values are strings like `"100px"`. Must remove `"px"` to do arithmetic. Use numeric-only strings to allow automatic type conversion to numbers. After calculations, concatenate `"px"` back before setting `style.top` and `style.left`. Placing JavaScript in a separate `.js` file prevents HTML comment issues.

Example issue:

If JavaScript is inside HTML comments and contains `--` (like `x--;`), HTML validators may flag it as an invalid comment.

The `setTimeout` call is complex because:

- It needs to pass updated values of `x` and `y`.
- The function call string must be built dynamically by concatenating the variable values into the string.

`moveText.html`

```
<!DOCTYPE html>
<!-- moveText.html
Uses moveText.js
Illustrates a moving text element
-->
<html lang = "en">
<head>
<title> Moving text </title>
<meta charset = "utf-8" />
<script type = "text/javascript"
src = "moveText.js">
</script>
</head>
<!-- Call the initializing function on load, giving the
destination coordinates for the text to be moved
-->
<body onload = "initText()">
<!-- The text to be moved, including its initial position -->
<p>
<span id = 'theText' style =
"position: absolute; left: 100px; top: 100px;
```

```
font: bold 1.7em 'Times Roman';
color: blue;"> Jump in the lake!
</span>
</p>
</body>
</html>
```

moveText.js

```

//*****
// This is moveText.js - used with moveText.html
var dom, x, y, finalx = 300, finally = 300;
// ***** //
// A function to initialize the x- and y-coordinates
// of the current position of the text to be moved
// and then call the mover function
function initText() {
  dom = document.getElementById('theText').style;
  /* Get the current position of the text */
  var x = dom.left;
  var y = dom.top;
  /* Convert the string values of left and top to
  numbers by stripping off the units */
  x = x.match(/\d+/);
  y = y.match(/\d+/);
  /* Call the function that moves it */
  moveText(x, y);
} /*** end of function initText */
// ***** //
// A function to move the text from its original
// position to (finalx, finally)
function moveText(x, y) {

  /* If the x-coordinates are not equal, move
  x toward finalx */
  if (x != finalx)
    if (x > finalx) x--;
    else if (x < finalx) x++;
  /* If the y-coordinates are not equal, move
  y toward finally */
  if (y != finally)
    if (y > finally) y--;
    else if (y < finally) y++;
  /* As long as the text is not at the destination,
  call the mover with the current position */
  if ((x != finalx) || (y != finally)) {
    /* Put the units back on the coordinates before
    assigning them to the properties to cause the
    move */
    dom.left = x + "px";
  }
}
```

```
dom.top = y + "px";  
/* Recursive call, after a 1-millisecond delay */  
setTimeout("moveText(" + x + "," + y + ")", 1);  
}  
} /*** end of function moveText */
```

Dragging and Dropping Elements

Drag-and-drop allows users to move elements around the screen. Drag-and-drop can be done entirely with DOM 0. But to make it work in all browsers, you'd need browser detection and different code for each. DOM 2 is preferred for cleaner, portable code. DOM 0 model: Simple but less portable. Example: inline onmousedown="grabber(event)". DOM 2 model: More modern and portable (addEventListener and removeEventListener). The example uses DOM 2 for most handlers to avoid browser-specific code differences.

Achieved by:

- Listening to mousedown, mousemove, and mouseup events.
- Changing the element's top and left properties to update its position.

Example: Magnetic Poetry System

- HTML + JavaScript example shows:
 - Two static lines of a poem.
 - A set of movable words for creating the last two lines.
- Uses a mix of event models:
 - DOM 0 for mousedown (inline call to handler).
 - DOM 2 for mousemove and mouseup handlers.

Main Event Handlers:

a) grabber (for mousedown)

- Takes the Event object as a parameter.
- Gets the element being moved using currentTarget.
- Stores the element in a global variable (for use by other handlers).
- Calculates:
 - The difference between element's position and mouse cursor's position.
 - These differences are stored in global variables to maintain correct movement during dragging.
- Registers:
 - mover handler for mousemove → moves element.
 - dropper handler for mouseup → ends drag.

b) mover (for mousemove)

- Uses stored differences to update element's top and left as the mouse moves.

c) dropper (for mouseup)

- Unregisters both mover and dropper handlers.
- Ends the drag-and-drop action.

dragNDrop.html

```
<!DOCTYPE html>
<!-- dragNDrop.html
An example to illustrate the DOM 2 Event model
Allows the user to drag and drop words to complete
a short poem.
Does not work with IE browsers before IE9
-->
<html lang = "en">
<head>
<title> Drag and drop </title>
<meta charset = "utf-8" />
<script type = "text/javascript" src = "dragNdrop.js" >
</script>
</head>
<body style = "font-size: 20px;">
<p>
Roses are red <br />
Violets are blue <br />
<span style = "position: absolute; top: 200px; left: 0px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> candy </span>
<span style = "position: absolute; top: 200px; left: 75px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> cats </span>
<span style = "position: absolute; top: 200px; left: 150px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> cows </span>

<span style = "position: absolute; top: 200px; left: 225px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> glue </span>
<span style = "position: absolute; top: 200px; left: 300px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> is </span>
<span style = "position: absolute; top: 200px; left: 375px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> is </span>
<span style = "position: absolute; top: 200px; left: 450px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> meow </span>
<span style = "position: absolute; top: 250px; left: 0px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> mine </span>
<span style = "position: absolute; top: 250px; left: 75px;
```

```

background-color: lightgrey;"
onmousedown = "grabber(event);"> moo </span>
<span style = "position: absolute; top: 250px; left: 150px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> new </span>
<span style = "position: absolute; top: 250px; left: 225px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> old </span>
<span style = "position: absolute; top: 250px; left: 300px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> say </span>
<span style = "position: absolute; top: 250px; left: 375px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> say </span>
<span style = "position: absolute; top: 250px; left: 450px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> so </span>
<span style = "position: absolute; top: 300px; left: 0px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> sticky </span>
<span style = "position: absolute; top: 300px; left: 75px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> sweet </span>
<span style = "position: absolute; top: 300px; left: 150px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> syrup </span>
<span style = "position: absolute; top: 300px; left: 225px;
background-color: lightgrey;"
    onmousedown = "grabber(event);"> too </span>

    <span style = "position: absolute; top: 300px; left: 300px;
background-color: lightgrey;"
onmousedown = "grabber(event);"> yours </span>
</p>
</body>
</html>

```

dragNDrop.js

```

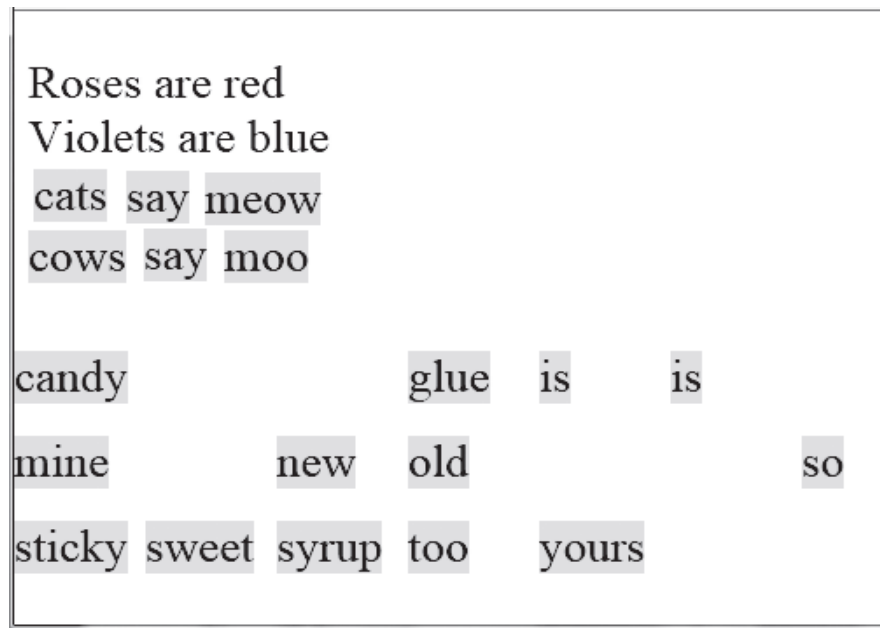
// dragNDrop.js
// An example to illustrate the DOM 2 Event model
// Allows the user to drag and drop words to complete
// a short poem.
// Does not work with IE browsers before IE9
// Define variables for the values computed by
// the grabber event handler but needed by mover
// event handler
var diffX, diffY, theElement;
// *****
// The event handler function for grabbing the word
function grabber(event) {

```

```
// Set the global variable for the element to be moved
theElement = event.currentTarget;
// Determine the position of the word to be grabbed,
// first removing the units from left and top
var posX = parseInt(theElement.style.left);
var posY = parseInt(theElement.style.top);
// Compute the difference between where it is and
// where the mouse click occurred
diffX = event.clientX - posX;
diffY = event.clientY - posY;
// Now register the event handlers for moving and
// dropping the word
document.addEventListener("mousemove", mover, true);
document.addEventListener("mouseup", dropper, true);

// Stop propagation of the event and stop any default
// browser action
event.stopPropagation();
event.preventDefault();
} /** end of grabber
// *****
// The event handler function for moving the word
function mover(event) {
// Compute the new position, add the units, and move the word
theElement.style.left = (event.clientX - diffX) + "px";
theElement.style.top = (event.clientY - diffY) + "px";
// Prevent propagation of the event
event.stopPropagation();
} /** end of mover
// *****
// The event handler function for dropping the word
function dropper(event) {
// Unregister the event handlers for mouseup and mousemove
document.removeEventListener("mouseup", dropper, true);
document.removeEventListener("mousemove", mover, true);
// Prevent propagation of the event
event.stopPropagation();
} /** end of dropper
```

Display of dragNDrop.html



Chapter 2

Welcome to React

Obstacles and Roadblocks

Understanding React components changes how you think about and approach web development. Components promote a modular, reusable, and structured coding style. Learning components is like opening a door to a new development perspective. Encourages breaking UI into smaller pieces that manage their own logic and rendering. Some obstacles exist before you can write production-ready React code.

These may include:

- Understanding JSX syntax.
- Grasping component state and props.
- Learning component lifecycle and hooks.
- Adapting to declarative programming.

React Is a Library

React library is small which handles only one part of the job: building UI components. Does not include all tools expected from a full JavaScript framework. Many decisions about: Which additional tools to use (e.g., routing, state management, build tools), How to structure the project; Developers must assemble their own toolchain from the ecosystem. New tools appear frequently & Older tools can quickly become obsolete. Constant new library names keep coming in steadily and frequently which can be hard to keep up.

New ECMAScript Syntax

React matured during a transformative period in JavaScript's history. This period was chaotic because of big changes in how JavaScript evolved.

ECMAScript Release Cycle Changes:

- Before 2015:
 - ECMAScript specs released very infrequently (sometimes once in 10 years).
 - Developers rarely had to learn new syntax.
- From 2015 onwards:
 - New features and syntax added every year.
 - Shift from numbered releases (e.g., ECMAScript 3, ECMAScript 5) to year-based naming (e.g., ECMAScript 2016, ECMAScript 2017).

Impact on React Developers:

- React community quickly adopts the newest JavaScript features.
- Documentation often assumes you know the latest ECMAScript syntax.
- Without knowledge of the latest JavaScript features, React code can be hard to follow.

Popularity of Functional JavaScript

JavaScript is not purely functional, but functional concepts can be applied in its code. More and more developers are using functional programming ideas in JavaScript. React emphasizes functional programming over object-oriented programming.

React encourages use of:

- Pure functions
- Immutability
- Composition

Better testability (functions are easier to test than classes with internal state). Potential performance improvements due to predictable behaviour.

React's Future

React is still relatively new. Core functionality is fairly stable, but changes can still occur because of reimplementation of React's core algorithm. Main goal is to increase rendering speed for updates and UI changes. Exact impact on developers is not yet fully known & many React changes are driven by the range of devices being targeted.

React isn't limited to web browsers:

- React Native (2015): Enables building iOS and Android native apps with React.
- React VR: Framework for building interactive, 360° virtual reality experiences using React + JavaScript.

Mastering React prepares you to:

- Build apps for different screen sizes and device types.
- Adapt to a changing ecosystem.
- Develop applications for platforms beyond the web browser.

Keeping Up with the Changes

React and related tools sometimes introduce breaking changes. Future updates may cause some example code (which are mentioned here) to stop working.

Code samples (which are mentioned here) will still work if you use the exact package versions specified in package.json & Install dependencies based on those version numbers.

Official React blogs posts detailed announcements for new versions including changelogs explaining updates and changes. If you can't attend conferences in person, Talks are often released on YouTube after the events.

There are also a variety of popular React conferences that you can attend for the latest React information. These include:

React Conf

Facebook-sponsored conference in the Bay Area

React Rally

Community conference in Salt Lake City

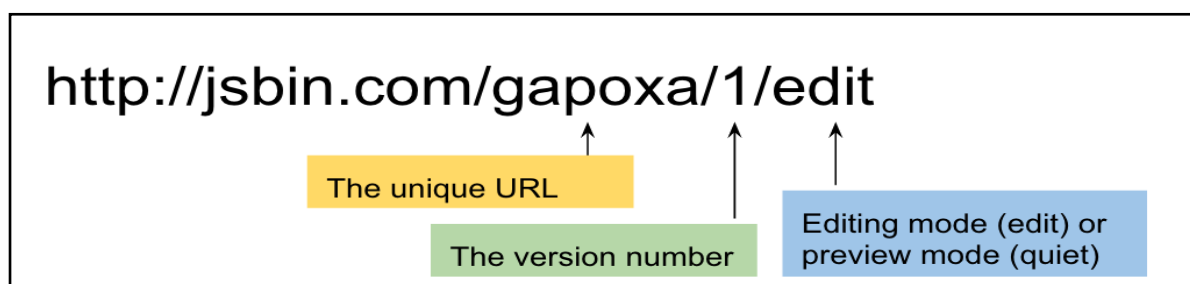
ReactiveConf

Community conference in Bratislava, Slovakia
React Amsterdam
Community conference in Amsterdam

Working with the Files

File Repository:

- A GitHub repository is provided for the book.
- Contains:
 - All code files organized by chapter.
 - JSBin samples for interactive editing.
- JSBin: Online code editor (similar to CodePen and JSFiddle).
- Main benefit:
 - Click a link → instantly start editing and experimenting with the code.
- When you edit or create a JSBin - Generates a unique URL for your code.
- JSBin URL structure:



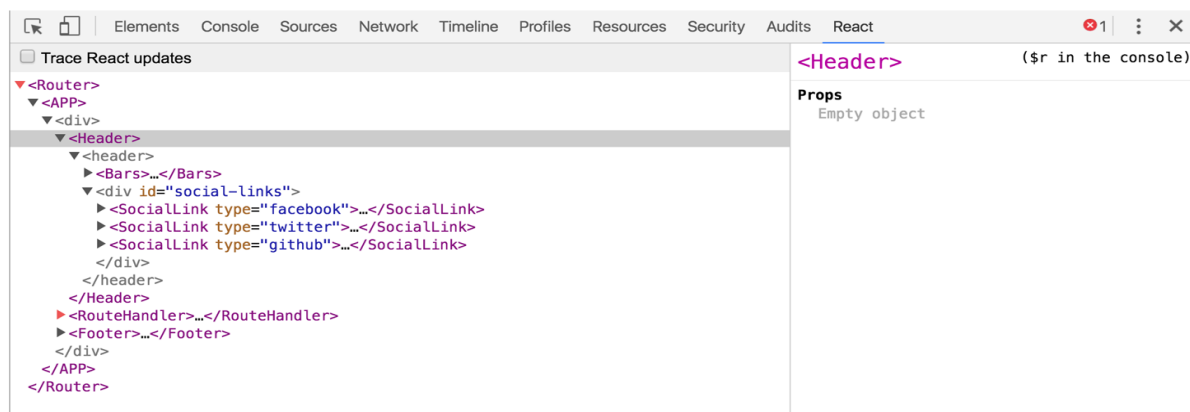
React Developer Tools:

There are several developer tools that can be installed as browser extensions or addons that you may find useful as well:

- **react-detector**
 - react-detector is a Chrome extension that lets you know which websites are using React and which are not.
- **show-me-the-react**
 - This is another tool, available for Firefox and Chrome, that detects React as you browse the internet.
- **React Developer Tools**
 - This is a plugin that can extend the functionality of the browser's developer tools. It creates a new tab in the developer tools where you can view React elements. If you prefer Chrome, you can install it as an extension; you can also install it as an add-on for Firefox.

Any time you see react-detector or show-me-the-react as active, you can open the developer tools and get an understanding of how React is being used on the site.

Viewing the React Developer Tools



Installing Node.js:

When working with Node and React, you will need to use the command line. On the Mac, this is called the Terminal. On a PC, it is called the Command Prompt. Run the following commands to check your current version on node and npm (Node Package Manager)

Instructions on how to install or upgrade are as follows.

First, check to see if you have Node.js installed:

```
$ node -v
```

If this returns a version number, Node.js is installed. If the command is not found, you'll need to install Node.js from the Node.js website (<https://nodejs.org/en/download>). Download the installer, run it, and follow the instructions.

Next, check your version of npm:

```
$ npm -v
```